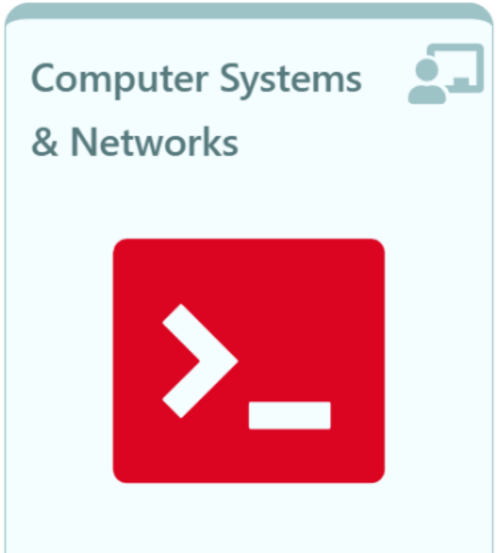
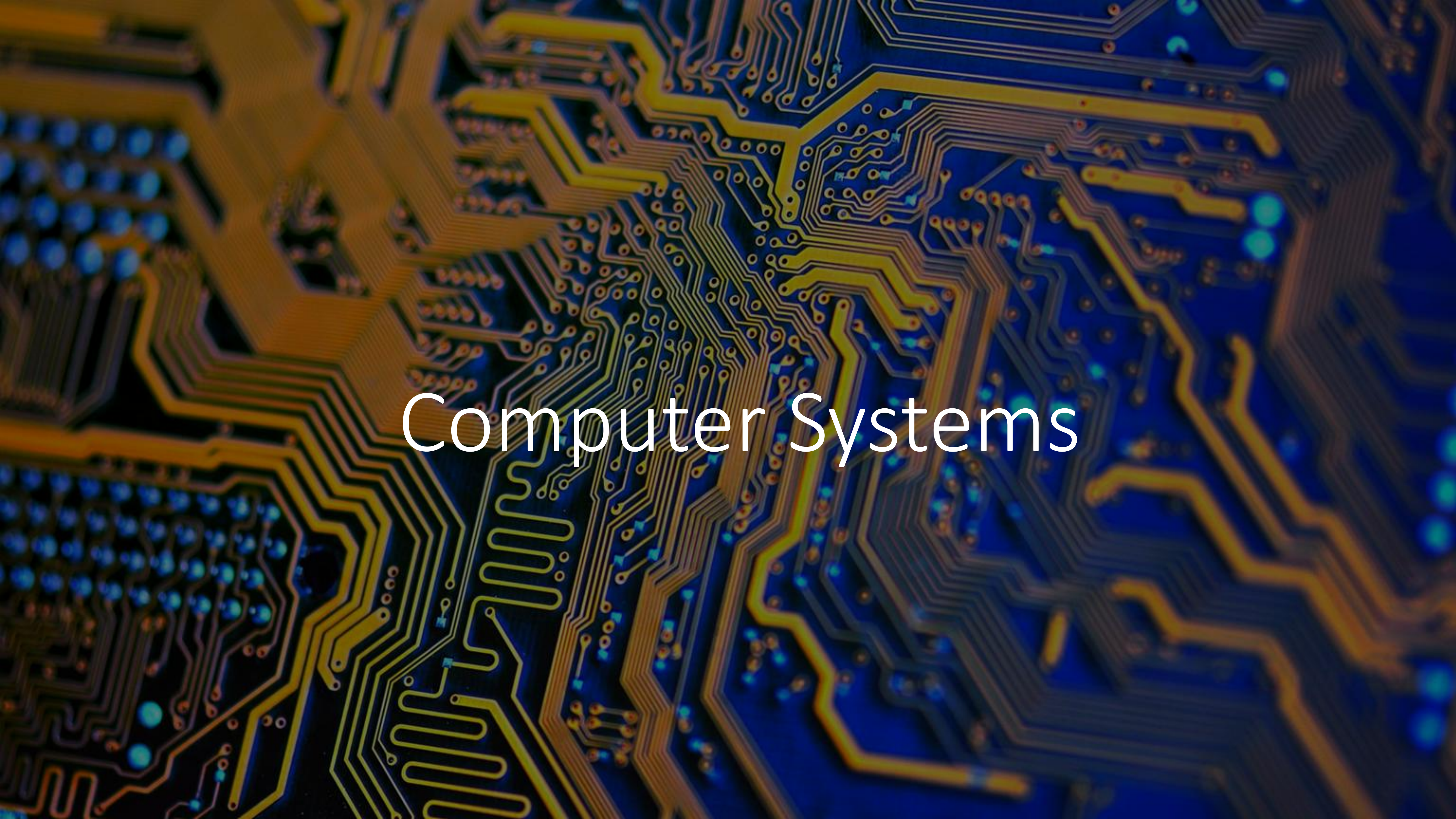
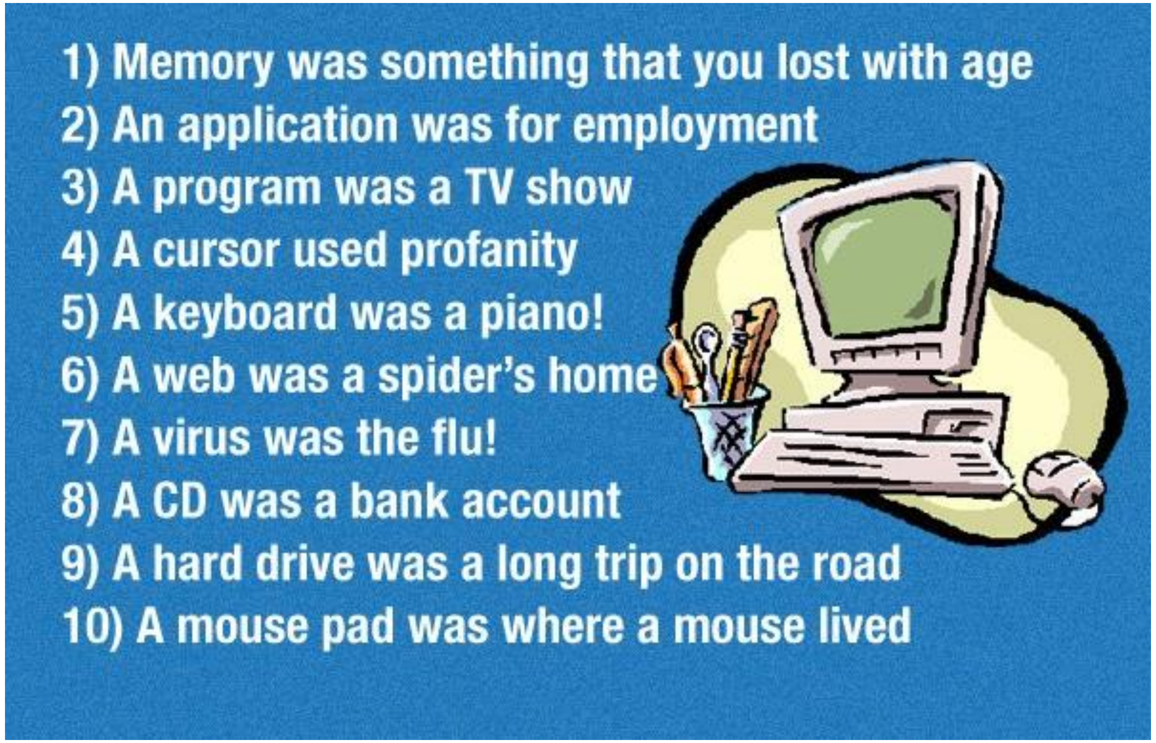
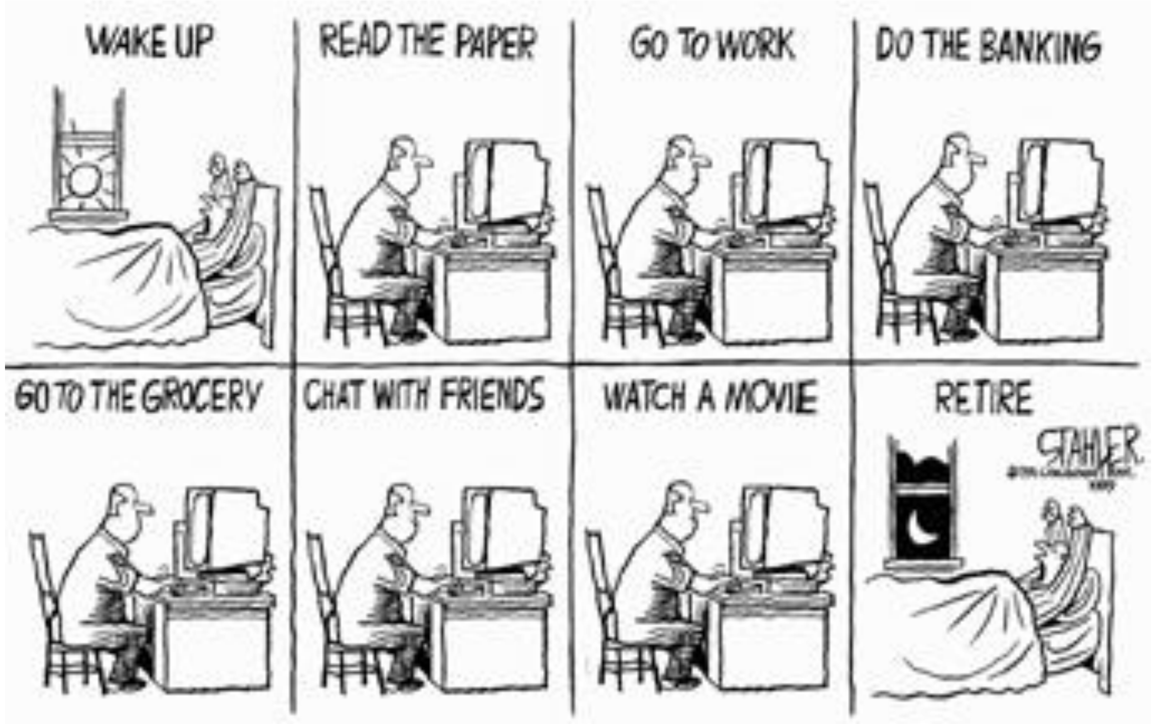
# Computer Systems & Networks



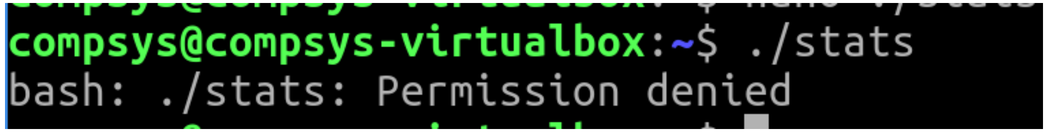


# Computer Systems


1) Memory was something that you lost with age
2) An application was for employment
3) A program was a TV show
4) A cursor used profanity
5) A keyboard was a piano!
6) A web was a spider's home
7) A virus was the flu!
8) A CD was a bank account
9) A hard drive was a long trip on the road
10) A mouse pad was where a mouse lived


WAKE UP
READ THE PAPER
GO TO WORK
DO THE BANKING
GO TO THE GROCERY
CHAT WITH FRIENDS
WATCH A MOVIE
RETIRE
STAHLER

# Number bases

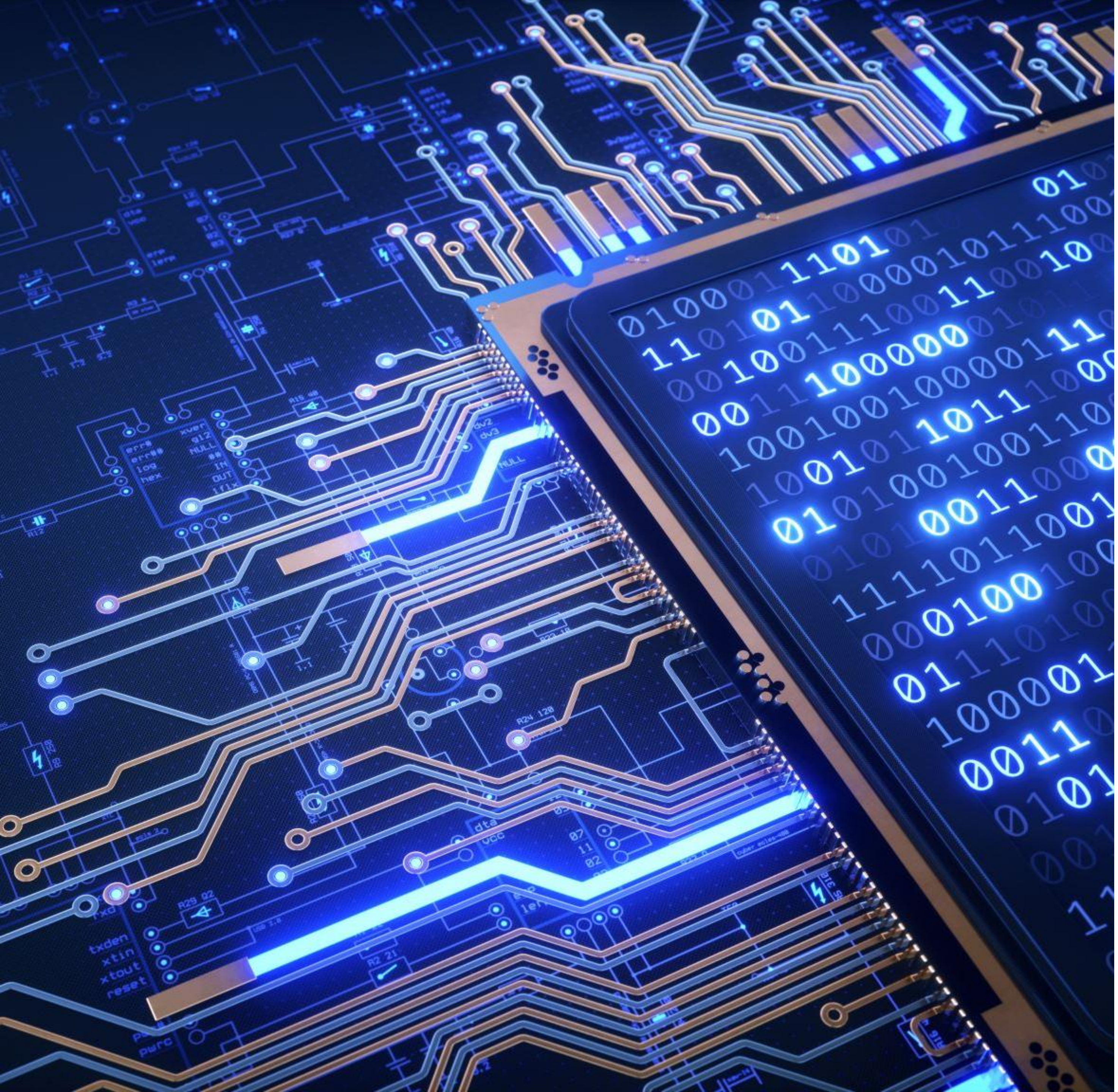



There are basically **four** major types of number systems, namely:

1. decimal (base10),
2. binary (base2),
3. octal (base8) and
4. hexadecimal (base16)

# Number Bases in Computer Systems

| Decimal (base 10) | Binary (base 2) | Octal (base 8) | Hexadecimal (base 16) |
|---|---|---|---|
| 00 | 0000 | 00 | 0 |
| 01 | 0001 | 01 | 1 |
| 02 | 0010 | 02 | 2 |
| 03 | 0011 | 03 | 3 |
| 04 | 0100 | 04 | 4 |

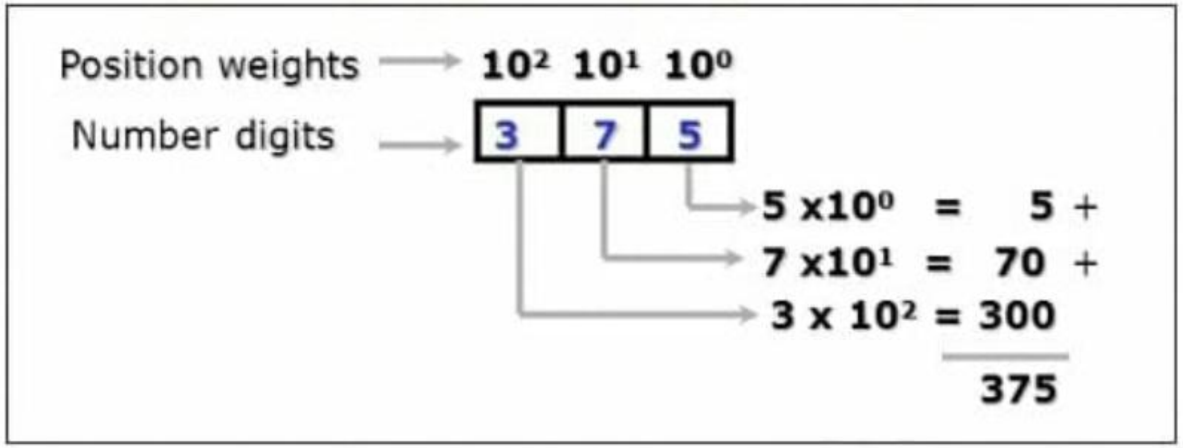


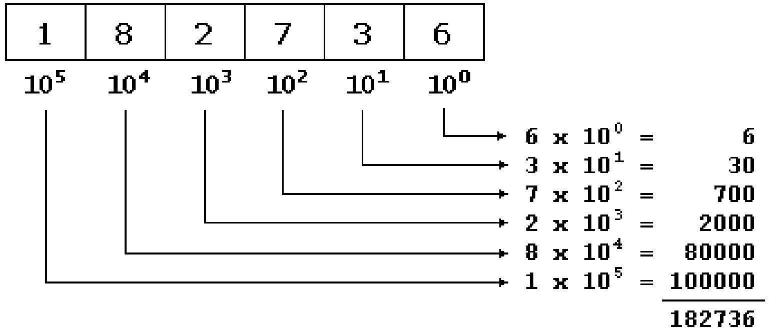
# 1 of 4:
# Decimal

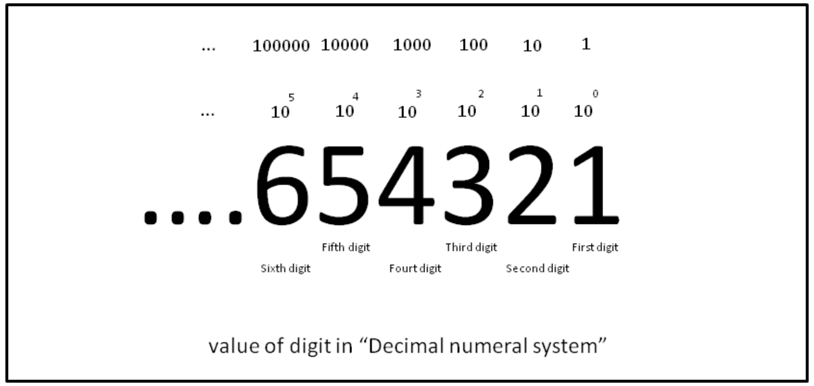
- Let’s analyze the decimal number **375**:

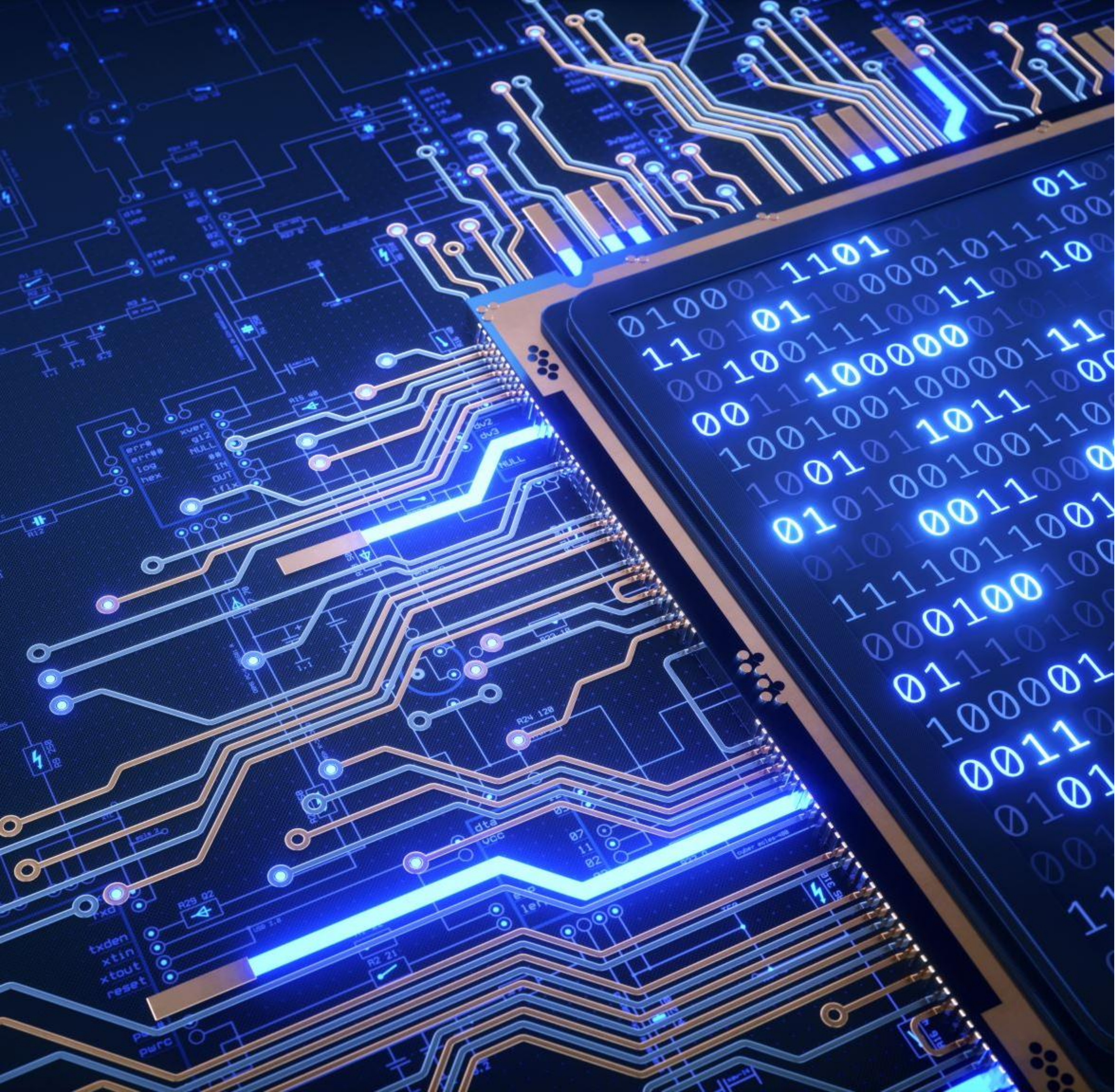
$$375 = (3 \times 100) + (7 \times 10) + (5 \times 1)$$

$$= (3 \times 10^2) + (7 \times 10^1) + (5 \times 10^0)$$





# 2 of 4:
# **Binary**

# 1 inputs: binary

- For 1 input there are $2^1$ possible outputs
- Represents either being on/off

| INPUT 1: Light | POSSIBLE OUTPUTS: |
|---|---|
| 0 | Light off |
| 1 | Light on |

# (1) NUMBER BASE: Binary

- a base-2 numeral system that uses two symbols (typically represented as) 0 and 1, to represent numeric values

| | Column 8 | Column 7 | Column 6 | Column 5 | Column 4 | Column 3 | Column 2 | Column 1 |
|---|---|---|---|---|---|---|---|---|
| Base$^{exp}$ | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| Weight | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

***Sample Storage amounts:***

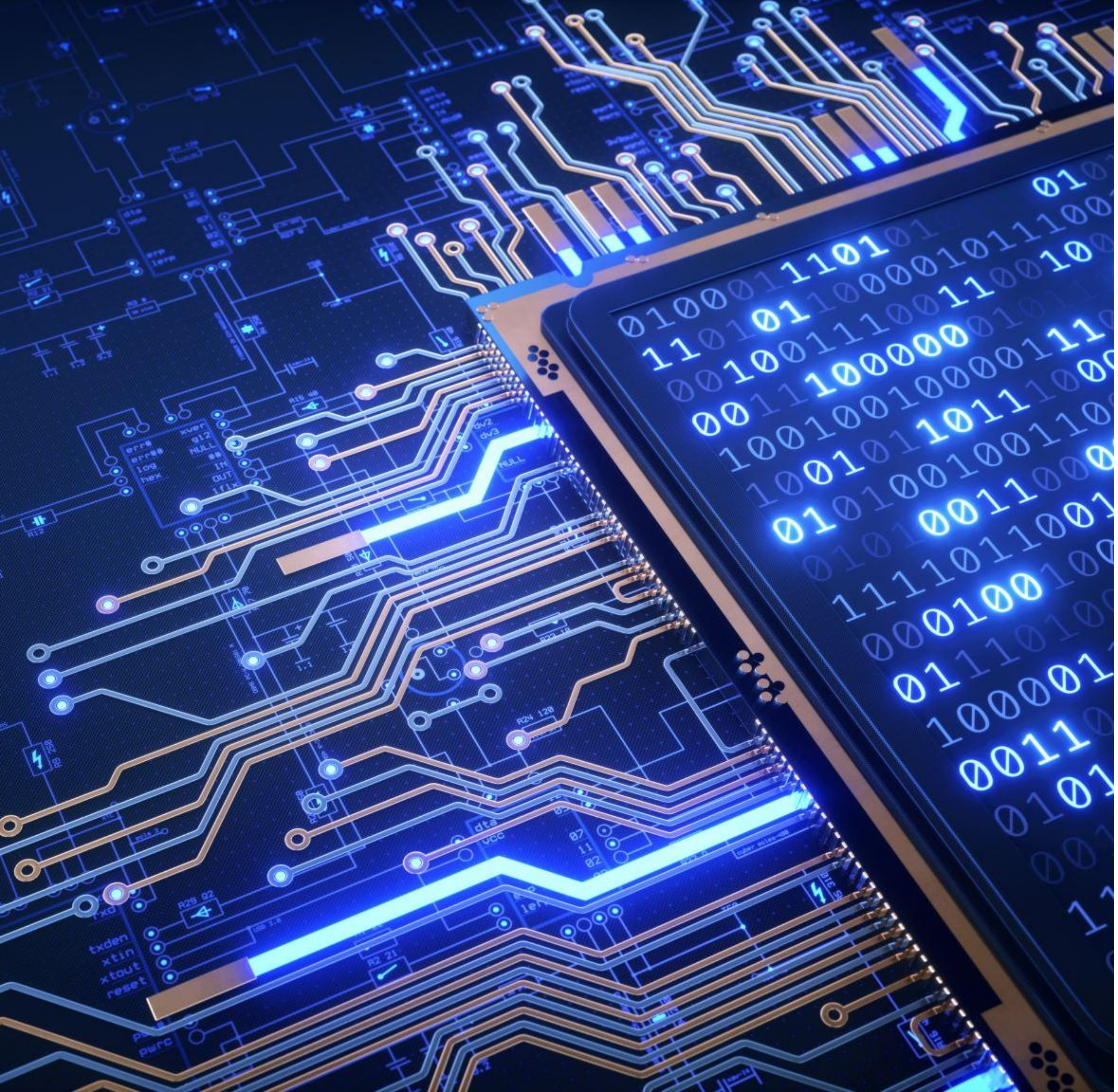
1 TB
512 GB
256 GB
128 GB
64 GB
16 GB
32 GB

| | Column 8 | Column 7 | Column 6 | Column 5 | Column 4 | Column 3 | Column 2 | Column 1 |
|---|---|---|---|---|---|---|---|---|
| Base$^{exp}$ | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| **Weight** | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

# EXER: Convert $101101_2$ to $\text{decimal}_{10}$

| Binary Number | Decimal Number |
|---|---|
| 0 | 0 |
| 1 | 1 |
| 10 | 2 |
| 11 | 3 |
| 100 | 4 |
| 101 | 5 |
| 110 | 6 |
| 111 | 7 |
| 1000 | 8 |
| 1001 | 9 |
| 1010 | 10 |
| 1011 | 11 |

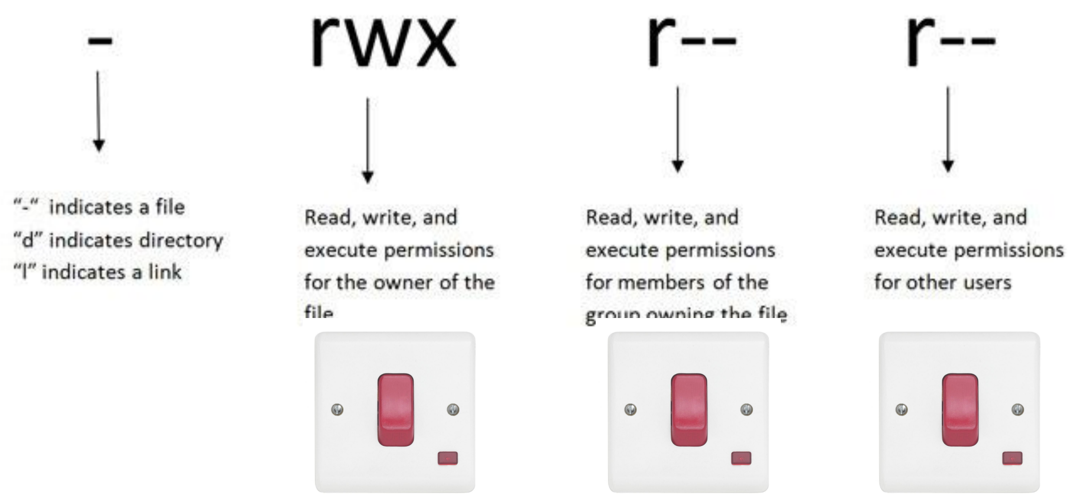
| | Column 8 | Column 7 | Column 6 | Column 5 | Column 4 | Column 3 | Column 2 | Column 1 |
|---|---|---|---|---|---|---|---|---|
| $\text{Base}^{\text{exp}}$ | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| Weight | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |



# 3 of 4: Octal

## Octal Representation

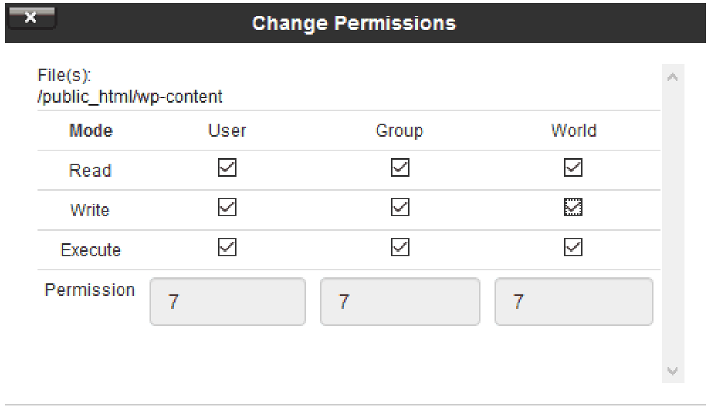
| | | | | | |
|---|---|---|---|---|---|
| 0 | 000 | - | - | - | No permissions |
| 1 | 001 | - | - | x | Only Execute |
| 2 | 010 | - | w | - | Only Write |
| 3 | 011 | - | w | x | Write and Execute |
| 4 | 100 | r | - | - | Only Read |
| 5 | 101 | r | - | x | Read and Execute |
| 6 | 110 | r | w | - | Read and Write |
| 7 | 111 | r | w | x | Read, Write and Execute |

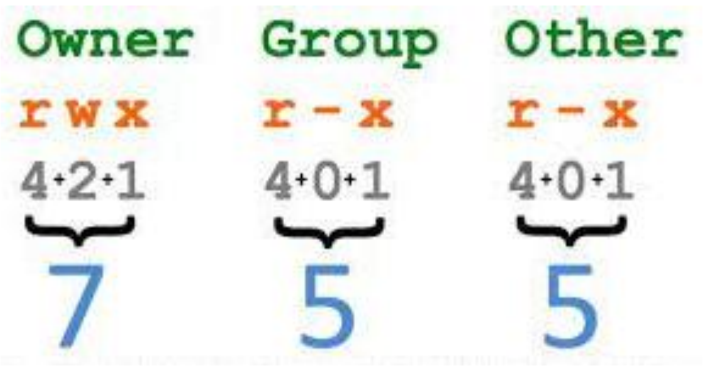



# (2) NUMBER BASE:
Octal

- base-8 numeral system
  - uses eight symbols, typically represented as digits 0 through 7, to represent numeric values
- Each digit's position in an octal number represents a power of 8
- distinct value of between $000_2(0_8)$ and $111_2(7_8)$ .... Why?

# File Permissions




| Binary | Octal |
|---|---|
| 0000 | 0 |
| 0001 | 1 |
| 0010 | 2 |
| 0011 | 3 |
| 0100 | 4 |
| 0101 | 5 |
| 0110 | 6 |
| 0111 | 7 |
| 1000 | 10 |
| 1001 | 11 |
| 1010 | 12 |
| 1011 | 13 |
| 1100 | 14 |

- In the octal representation, each permission is represented by a digit:

4 for Read,

2 for Write, and

1 for Execute

***Why?***

| | | | |
|---|---|---|---|
| Octal | ***7*** | ***5*** | ***5*** (base8) |
| ≡Binary | **111** | **101** | **101** (base2) |
| | Owner | Group | Others |

# Show that $7_8 \equiv 7_{10}$

| Weighting | $8^3$ | $8^2$ | $8^1$ | $8^0$ |
|---|---|---|---|---|
| = | 512 | 64 | 8 | 1 |
| | | | No. to Convert: | $7_8$ |
| | | | | $+(7*8^0)$<br>$+7*1$<br>$+7$ |
| | | | | $7_8=7_{10}$ |

# Show that $75_8 \equiv 61_{10}$

| Weighting | $8^3$ | $8^2$ | $8^1$ | $8^0$ |
|---|---|---|---|---|
| = | 512 | 64 | 8 | 1 |
| **No. to Convert:** | | | $\mathbf{7}_8$ | $\mathbf{5}_\mathbf{8}$ |
| | | | $(7*8^1)$<br>$=7*8$<br>$=56$ | $+(5*8^0)$<br>$+5*1$<br>$+5$ |
| | | | $75_8=61_{10}$ | |

**RECALL**

# Number Bases in Computer Systems

| Decimal (base 10) | Binary (base 2) | Octal (base 8) | Hexadecimal (base 16) |
|---|---|---|---|
| 00 | 0000 | 00 | 0 |
| 01 | 0001 | 01 | 1 |
| 02 | 0010 | 02 | 2 |
| 03 | 0011 | 03 | 3 |
| 04 | 0100 | 04 | 4 |

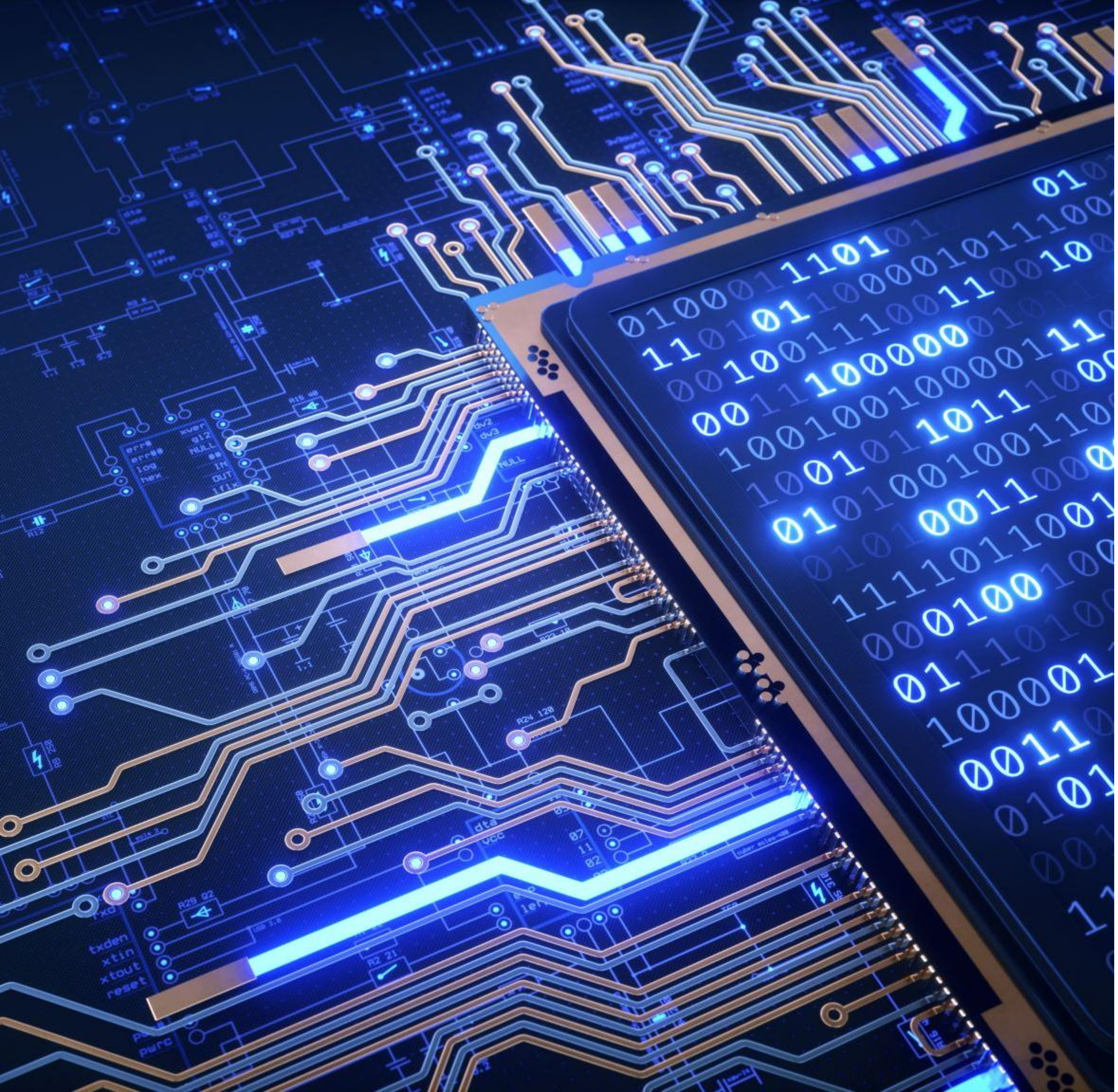
# chmod

is used to change permissions of a file

**CHMOD is used to change permissions of a file.**

| PERMISSION | | | COMMAND |
|---|---|---|---|
| U | G | W | |
| rwx | rwx | rwx | chmod 777 filename |
| rwx | rwx | r-x | chmod 775 filename |
| rwx | r-x | r-x | chmod 755 filename |
| rw- | rw- | r-- | chmod 664 filename |
| rw- | r-- | r-- | chmod 644 filename |
| User | Group | World | |

r = Readable
w = Writable
x = Executable
- = None



# 4 of 4:
# Hexadecimal

# (3) NUMBER BASE: Hexadecimal

- Longer binary numbers are what computers use at the hardware level which is not human-user-friendly.

- Hex uses 16 symbols to represent its value:

| Hexadecimal Symbol | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Decimal Value | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |

# IPv4 addresses use Hex

An IPv4 address like: 192.168.1.1 is actually just a **32-bit number**.

Those bits are grouped into 4 bytes (8 bits each).

Each byte:

- is stored in **binary**
- is often represented internally or in tools as **hex**
- is shown to humans as **decimal**

# …IPv4 addresses use Hex

192.168.1.1 IPv4 address becomes:

C0.A8.01.01 in hex

Hex is useful because:

- **1 hex digit = 4 bits**, so it maps cleanly to binary
- It’s **shorter and more readable** than long binary strings

# IPv6 addresses use Hex

An IPv6 address like:

85a3:0db8:85a3:0000:0000:8a2e:0370:7334

[you’ll do more later in sem]

- written **directly in hex**
- grouped into 16-bit chunks

8 groups X 16 bits = 128 bits number

Trying to write that in decimal would be… chaos.

RECALL

# Number Bases in Computer Systems

| Decimal (base 10) | Binary (base 2) | Octal (base 8) | Hexadecimal (base 16) |
|---|---|---|---|
| 00 | 0000 | 00 | 0 |
| 01 | 0001 | 01 | 1 |
| 02 | 0010 | 02 | 2 |
| 03 | 0011 | 03 | 3 |
| 04 | 0100 | 04 | 4 |

# Web colour codes use Hex

| Color Name | Color Code |
|---|---|
| Red | #FF0000 |
| Cyan | #00FFFF |
| Blue | #0000FF |
| DarkBlue | #0000A0 |

# In Summary

# Octal is:

… a concise way to set and view Linux file permissions numerically

- where each digit represents the access level for:

1. owner,
2. group, and
3. others

respectively.

# Three categories of file user

Owner = permissions represented in the first octal digit

2. Group = permissions represented in the second octal digit

3. Others = permissions represented in the third octal digit

# Each set has **three** types of permissions

**r** (read) = 4

**w** (write) = 2

**x** (execute) = 1

These values are **added together** for each set

So a permission:

rwxr-xr–

becomes:

| | |
|---|---|
| Owner: | rwx = 7 |
| Group: | r-x = 5 |
| Others: | r-- = 4 |

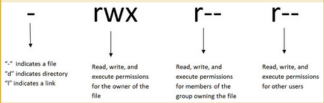
**Octal: 754(base 8)** written as $754_8$

# Try out today's lab

01: Computer Systems


NetAcad · Linux as an OS

02: File Permissions




Numbering Systems · Permissions · Octal

Lab


Fundamentals · Commands · Scripts · Permissions

- File Hierarchy and Navigation
- **Creating Files, Absolute and Relative Filenames**
- Navigation Basics
- Redirection
- File Manipulation
- Executable Scripts
- Week 1 Summary
- Cisco NetAcad